I used Karpathy's LLM Wiki to build a knowledge base that maintains itself with AI

12 min read · Apr 7, 2026

Balu Kosuri Follow

Listen Share ••• More

Here is a scenario most knowledge workers know too well.

You have 15 documents about a project. A product spec here. A meeting transcript is there. A competitor report is saved somewhere in your downloads. Three months later, someone asks you a simple question, and you spend an hour digging through folders trying to find that one paragraph you read once.

The documents exist. The knowledge is in there. But it is scattered, disconnected, and impossible to search without re-reading everything.

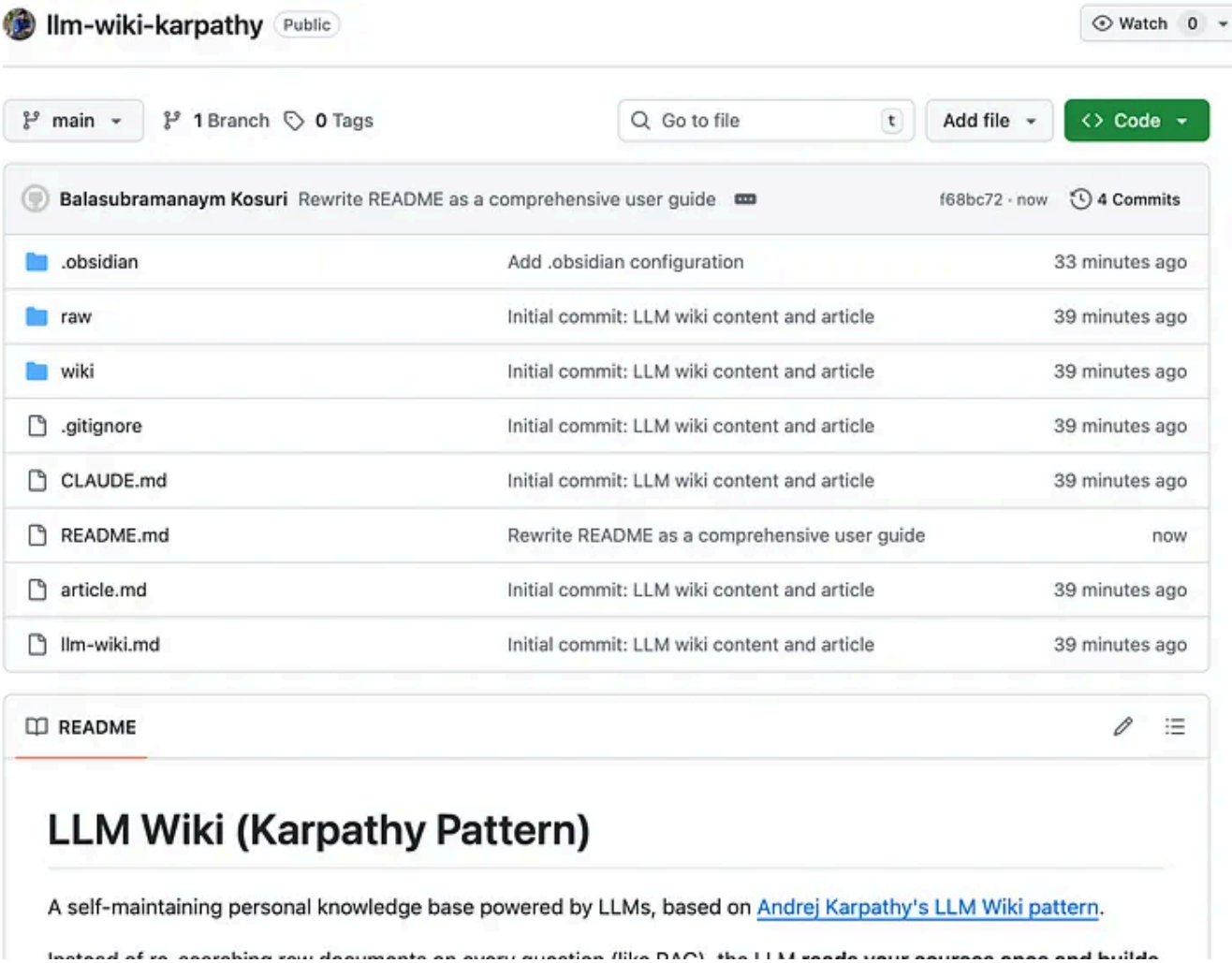
I found a solution. It involves a one-page idea document by Andrej Karpathy, an AI code editor called Cursor, and a free note-taking app called Obsidian. In about 30 minutes, I went from that one-page idea to a fully working personal knowledge base, ready to accept any document I throw at it and turn it into structured, interlinked wiki pages.

And I didn't write a single wiki page myself.

This article tells the story of how it happened and gives you a repo you can clone and start using today.

I USED
KARPATHY'S LLM
WIKI
TO BUILD A
KNOWLEDGE BASE
THAT MAINTAINS
ITSELF WITH AI

Get Started



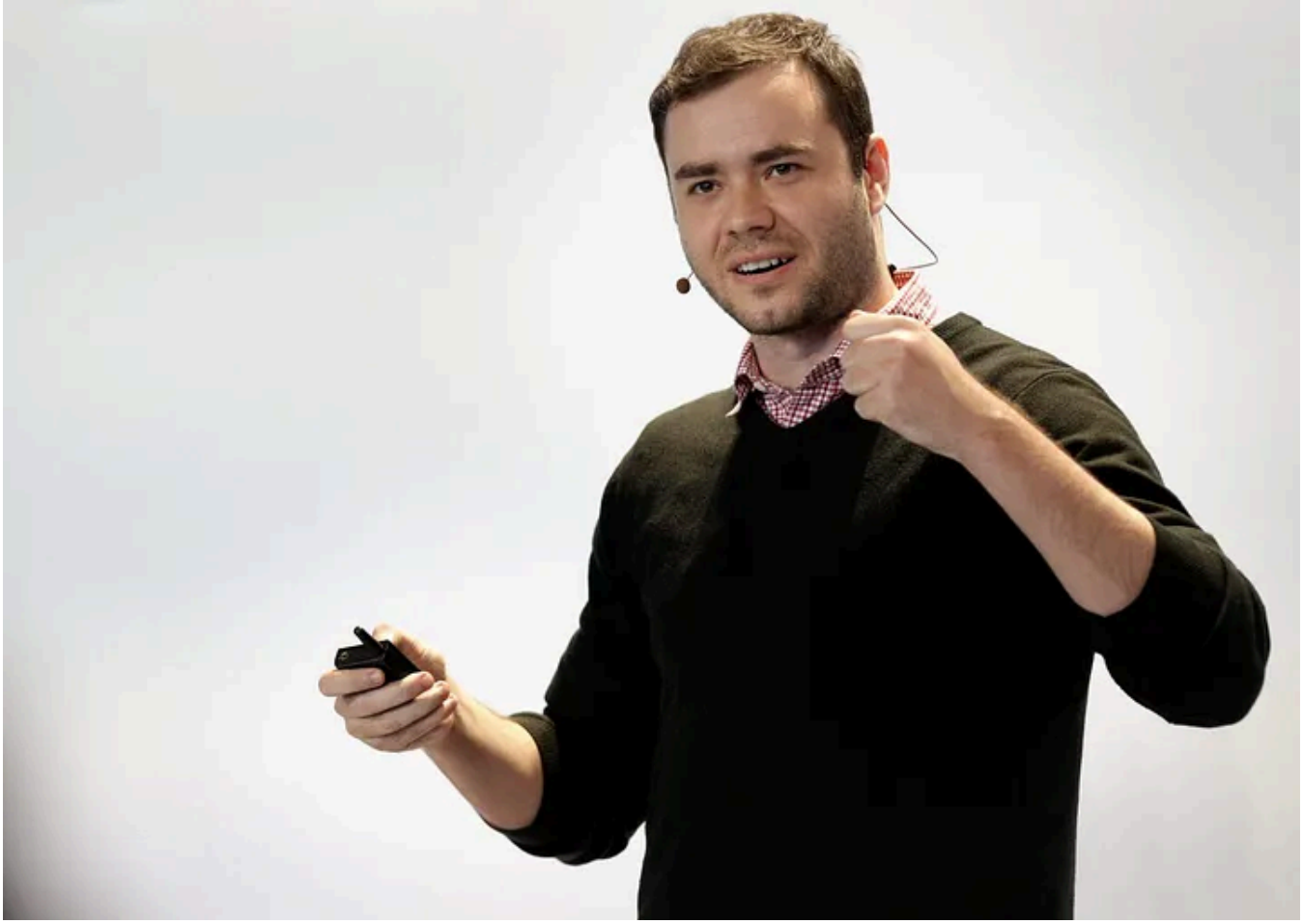
Repository link: https://github.com/balukosuri/llm-wiki-karpathy

What's included:

- Karpathy's original `llm-wiki.md` idea document
- A `CLAUDE.md` schema tailored for technical writers (edit it for your domain)
- Pre-configured Obsidian vault settings (graph view, hotkeys, sidebar)
- An empty `raw/` folder ready for your first source
- A `wiki/` folder with four starter pages (index, log, overview, glossary)

Drop your first document. Say "ingest." Watch the wiki write itself.

Who Is Andrej Karpathy?



Andrej Karpathy is one of the most well-known people in AI. He was a founding member at OpenAI, led the AI and Autopilot vision team at Tesla, and is known for explaining deep technical ideas in a way that anyone can follow. When Karpathy shares something, the AI community listens.

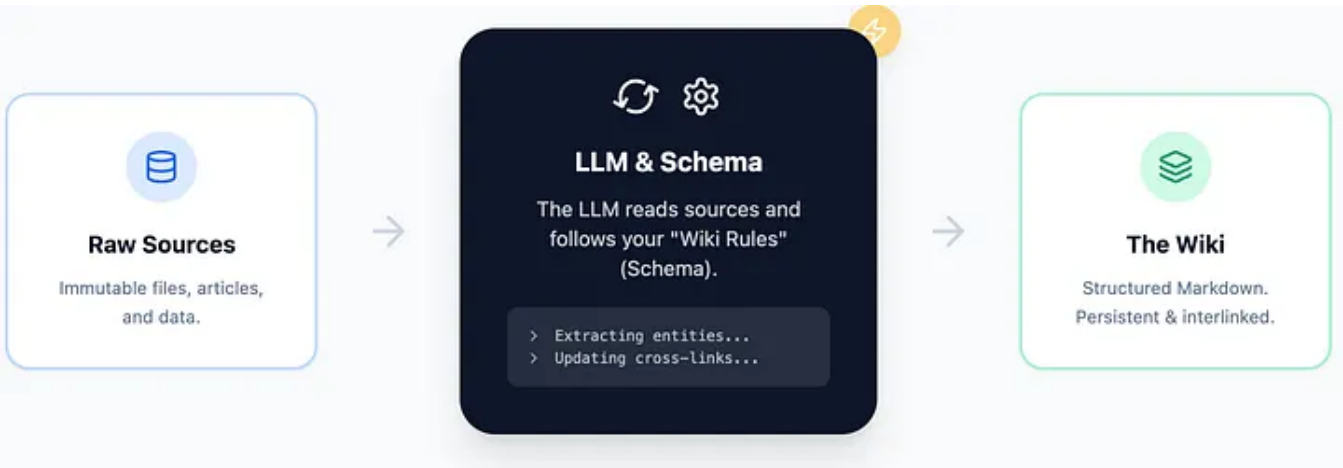
A few days ago, he shared a short document called `llm-wiki.md`. It was not a product or an app. It was just an idea — written in plain markdown describing a pattern for how to use AI agents to build and maintain personal knowledge bases.

The document was designed to be copy-pasted into any AI agent (Claude, ChatGPT, Codex, or others). The agent would read it, understand the pattern, and then build out a working version tailored to your needs.

Link to the original: Karpathy's llm-wiki.md

That one-page idea is the foundation of everything in this repo.

What Is LLM Wiki and How Does It Work?



The core idea is simple.

Most AI tools work like this: you upload documents, ask a question, and the AI searches through your files to generate an answer. This works fine , but the AI forgets everything after each question. The next time you ask something, it starts from scratch. It re-reads, re-searches, and re-derives the answer. Nothing is saved. Nothing builds on what came before.

LLM Wiki flips this around. Instead of searching your raw documents every time, the AI reads your documents once and builds a structured wiki from them. The wiki is a collection of markdown files, summary pages, product pages, concept pages, persona pages, and comparison tables, all interlinked with wiki-style links. When you add a new document, the AI doesn't start over. It reads the new source and updates the existing wiki, adding to pages that already exist, creating new ones where needed, flagging contradictions, and keeping everything consistent.

The wiki is the persistent artifact. It compounds over time. The more sources you feed it, the richer and more connected it gets.

— — — —

index. You browse it; the AI writes it.

3. **The schema** — A single file called `CLAUDE.md`. This is the instruction manual for the AI. It defines what types of pages exist, what workflow to follow when processing a new source, how to format pages, and when to check the wiki for problems. Think of it as the rulebook that turns a general-purpose AI into a disciplined wiki maintainer.

Three operations

Ingest: You drop a document into `raw/` and tell the AI to process it. The AI reads it, creates summary pages, updates entity pages across the wiki, adds new terms to the glossary, updates the index, and logs what it did. A single source can touch 10-15 wiki pages.

Query: You ask questions. The AI reads the wiki (not the raw files) to put together answers. Good answers can be saved back into the wiki as analysis pages, so your questions make the knowledge base richer over time.

Lint: You ask the AI to health-check the wiki. It finds contradictions, stale information, orphan pages with no links, and missing cross-references. Think of it as spell-check for your knowledge base.

How I built it with Cursor in three prompts

Here is exactly what happened. I opened Cursor (an AI-powered code editor), dropped Karpathy's `llm-wiki.md` file into an empty project folder, and started talking to the AI.

Prompt 1: "What is this and how can I make use of this as a technical writer?"

Cursor read the entire document and mapped the idea to my role:

Pain point	How LLM Wiki solves it
Product updates scattered across docs, Slack, and email	Ingest them all, the AI merges them into one wiki
Glossary that nobody maintains	The AI builds and updates a living glossary automatically
Onboarding to a new product or codebase	Feed it specs and docs, get a synthesized wiki
Competitor research done once and lost	The AI maintains a structured comparison that grows over time
Writing release notes from meeting recordings	Ingest transcripts, the AI files key decisions into existing pages

Pain pointHow LLM Wiki solves itProduct updates scattered across docs, Slack, and emailIngest them all, the AI merges them into one wikiGlossary that nobody maintainsThe AI builds and updates a living glossary automaticallyOnboarding to a new product or codebaseFeed it specs and docs, get a synthesized wikiCompetitor research done once and lostThe AI maintains a structured comparison that grows over timeWriting release notes from meeting recordingsIngest transcripts, the AI files key decisions into existing pages

Prompt 2: “Can you make a plan and create?”

Five words. Cursor planned and built the entire project in one pass:

- Created `raw/` and `wiki/` folders
- Wrote `CLAUDE.md` with entity types, page formats, a 9-step ingest workflow, query workflow, lint workflow, and session start checklist
- Created four starter wiki pages: `index.md`, `log.md`, `overview.md`, and `glossary.md`

Prompt 3: “Can you set up Obsidian?”

Cursor installed Obsidian via Homebrew and pre-configured the vault:

- New files land in `wiki/` by default
- Graph view color-coded by page type
- Keyboard shortcuts for graph view, search, and quick switching

- Overview page opens on launch

Two windows side by side: Cursor on the left for talking to the AI, Obsidian on the right for browsing the wiki in real time.

What You Get When You Clone This Repo

Here is the exact file structure:

```
project-root/
│
├── llm-wiki.md              # Karpathy's original idea document
├── CLAUDE.md                 # Schema — tells the AI how the wiki works
│
├── raw/                      # Your source documents (AI reads, never writes)
│   └── .gitkeep
│
├── wiki/                     # AI-generated knowledge base
│   ├── index.md              # Master catalog of all pages (empty, ready to fi
│   ├── log.md                # What happened and when
│   ├── overview.md           # Big-picture synthesis (evolves over time)
│   ├── glossary.md           # Terms, definitions, style rules
│   └── sources/              # One summary per raw document
│
└── .obsidian/                # Pre-configured Obsidian vault
    ├── app.json              # File paths, link behavior
    ├── appearance.json       # Theme, font size
    ├── core-plugins.json     # Which plugins are active
    ├── graph.json            # Graph view colors and layout
    ├── hotkeys.json          # Keyboard shortcuts
    └── workspace.json        # Default tabs and sidebar layout
```

Why this structure works:

- Clear separation. `raw/` is yours. `wiki/` is the AI's. You never write in `wiki/`. The AI never changes `raw/`.
- The schema is the brain. `CLAUDE.md` defines entity types, page formats, and workflows. The AI reads this file first and follows its rules. Edit this file to change how the AI behaves for your specific domain.

- The index is the map. When you ask a question, the AI reads `index.md` first to find relevant pages, then drills into them. No vector databases or embeddings needed — the index works surprisingly well up to hundreds of pages.
- The log is the timeline. Every ingest, query, and lint pass is recorded with timestamps. You always know what happened and when.
- Obsidian is pre-configured. Anyone who clones the repo gets a ready-to-use Obsidian vault with graph view, hotkeys, and sidebar layout already set up. No manual configuration needed.

How to Use This Repo

Step 1: Clone it

```
git clone [YOUR-REPO-URL]
cd llm-wiki
```

Step 2: Open in Cursor

Open the project folder in Cursor. The AI reads `CLAUDE.md` automatically and understands the wiki structure and all its rules.

If you use a different AI agent (Claude Code, Codex, or others), paste the contents of `CLAUDE.md` into your agent's context.

Step 3: Open in Obsidian

Open the same folder as an Obsidian vault. If you don't have Obsidian, just ask Cursor: "Set up Obsidian for me." It will install it and open the vault.

Everything is pre-configured — hotkeys, graph view colors, sidebar layout.

Step 4: Drop a source into raw/

Any document works:

- A product spec or design doc

- A meeting transcript
- A clipped web article (use Obsidian Web Clipper browser extension)
- A style guide
- A PDF report
- An email thread saved as text

Step 5: Say "ingest"

Type this in Cursor:

> "Ingest raw/my-document.pdf"

The AI will:

1. Read the document
2. Discuss key takeaways with you
3. Create a source summary page in `wiki/sources/`
4. Create new pages for any products, features, personas, or concepts it finds
5. Update the glossary with new terms
6. Update the index with all new pages
7. Update the overview if the big picture shifted
8. Log everything in `wiki/log.md` with a timestamp

You watch the pages appear in Obsidian in real time.

Step 6: Ask questions

> "What are the main risks identified across all my sources?"

The AI reads the wiki, puts together an answer, and asks: "Should I save this as a wiki page?" If you say yes, the answer becomes a permanent analysis page in the wiki. Your questions make the knowledge base richer.

Step 7: Keep feeding it

Every new source builds on what came before. The overview page evolves. The glossary grows. The cross-references multiply. After 10–15 sources, the wiki starts showing you connections you hadn't noticed.

Step 8: Lint occasionally

Every 10 ingests or so:

> "Lint the wiki"

The AI checks for:

- Contradictions between pages
- Stale claims that newer sources have replaced
- Orphan pages with no links pointing to them
- Important concepts mentioned but missing their own page
- Inconsistent terminology across pages

It reports what it found and asks which fixes to apply.

Who this is for

Technical writers — Every spec updates the glossary. Every customer call adds to persona pages. Every competitor analysis builds on the last.

Researchers — Papers, articles, and reports get filed, summarized, and cross-referenced. By project end you have a wiki with an evolving thesis and all connections made.

Product managers — Feed it PRDs, customer interviews, competitive analyses, and sprint retros. The wiki maintains the big picture.

Students — Each textbook chapter becomes a source. The AI builds concept pages and links them. By exam time you have a connected study guide.

Anyone building knowledge over time — Trip planning, hobby research, health tracking, course notes. Anything where information comes from multiple sources.

Example: a technical writer's first week

Day 1

Drop three onboarding docs into `raw/` (PRD, internal FAQ, release notes). Ingest each one. The AI creates product pages, persona pages, a glossary, and flags contradictions between documents. End of day: 8 to 10 wiki pages without writing a single one.

Open in app ↗

Medium

conflicts with the PRD. You now have a specific list of things to clarify.

Day 3

Clip three competitor docs with Obsidian Web Clipper. Ingest all three. The AI creates a comparison analysis. Ask it to draft a documentation outline based on the wiki. Save the outline as an analysis page.

Day 4

Open `wiki/glossary.md` before writing. Every term, spelling, and deprecated name is there. Check persona pages for audience. Check the product page for accuracy. Write from the wiki instead of digging through original files.

Day 5

Get review feedback. Save it as a markdown file, drop it in `raw/`, ingest. The AI renames the feature everywhere, moves the old name to the deprecated list, and updates every page that referenced it. One ingest, every page updated.

After one week: 15 to 20 wiki pages, a living glossary, an overview with open questions, a full activity log, and a graph view showing how everything connects.

Tips That Made It Work Better

Ingest one source at a time. You could batch-ingest many documents at once, but you lose the chance to guide the AI. Stay involved — read the summaries, tell the AI

what to emphasize, ask follow-up questions during ingestion. The wiki gets better when you participate.

Save your best questions. When you ask a question and get a useful answer, tell the AI to save it as an analysis page. Your explorations should compound in the wiki, not disappear into chat history.

Use graph view. Press `Cmd+G` in Obsidian often. The visual map shows which pages are hubs, which are isolated, and how everything connects. It is the most satisfying way to see your wiki grow.

Edit the schema. `CLAUDE.md` is not set in stone. If you discover you need a new page type for your domain (like "API endpoints" or "customer segments" or "recipe variations"), add it to the schema and tell the AI. The wiki adapts to your needs.

Check the glossary before writing. Every time you sit down to write something, open `wiki/glossary.md` first. It has the right terms, the wrong terms, and the reasons behind each choice. This keeps your writing consistent without you having to remember everything.

Don't write wiki pages yourself. Resist the temptation. Your job is to find good sources and ask good questions. The AI's job is the summarizing, cross-referencing, filing, and bookkeeping. Let it do its job.

Conclusion

The reason people abandon wikis is not that they stop caring about the knowledge. It is that the maintenance becomes too much.

Think about it. Updating cross-references. Keeping summaries current. Making sure page 7 doesn't contradict page 23. Adding new terms to the glossary. Linking new pages to old ones. This work is boring, repetitive, and never-ending. So the wiki goes stale. People stop trusting it. Eventually nobody opens it.

AI changes this equation completely.

The AI never gets tired of maintenance. It can update 15 files in a single pass. It notices when new information contradicts old claims. It keeps the glossary current

and the index complete and the cross-references up to date. The cost of wiki maintenance drops to nearly zero.

That is the insight behind Karpathy's idea. The hard part of a knowledge base was never the reading or the thinking. It was always the bookkeeping. And bookkeeping is exactly what AI is best at.

Your job becomes the interesting part: finding good sources, asking the right questions, and deciding what matters. Everything else — the grunt work that killed every wiki you've ever tried to maintain — is handled.

> *Karpathy mentions in his original document that this idea is related to Vannevar Bush's Memex from 1945 — a vision of a personal knowledge store with "associative trails" between documents. Bush imagined a machine that could follow links between related ideas, building a web of connected knowledge that grew richer with every use.*

The web we ended up with is nothing like that. It is public, noisy, and the connections between documents are mostly accidental.

Bush's vision was private, curated, and deeply personal. The LLM Wiki is closer to what he imagined than anything we've built in 80 years. The part Bush couldn't solve was who does the maintenance. Now we have an answer.

My name is Balasubramanyam Kosuri, and I work as a Technical writer. Connect with me on LinkedIn, for more such content.

AI | Knowledge Base | Andrej Karpathy | Documentation | Search

Follow

Written by Balu Kosuri

347 followers · 23 following

Just keep writing

Responses (16)

Christian Broberg

What are your thoughts?

D. M. Wen
Apr 12

Excellent resources for learning how to build up the personal knowledge management, in the step by step manner extremely clearly and a well structured walkthrough

72 1 reply Reply

Tony G
3 days ago

Great stuff. Thanks for the insight. Added to my "AI Favorites" list.

Will definitely look into trying this, but I'm not an Obsidian guy. I'll use a different wiki or doc system as an alternate UI, and generate mind-map files, maybe for XMind. I... more

9 1 reply Reply

Yesara she/her
Apr 12

Yeah his document is really thought through. I used it to build a agent wiki, they can connect to it via API . Humans can read it too and add articles if needed. it's SweetSophia/noosphere on github. I discovered his document after the work on the project started and applied his ideas to it midway. Really helpfull

3 2 replies Reply

See all responses

More from Balu Kosuri

Balu Kosuri

I Turned Andrej Karpathy's Autoresearch Into a Universal Skill

By Balasubramanyam Kosuri

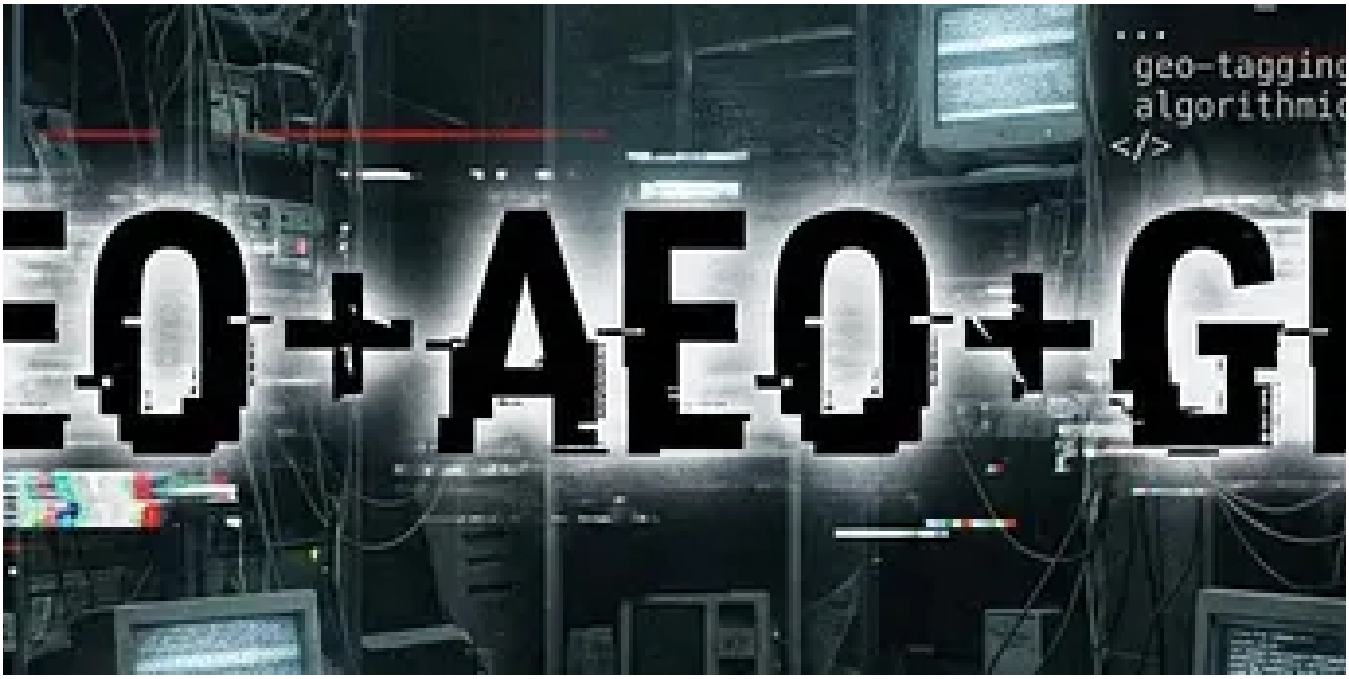
Mar 21 504 9

Balu Kosuri

Competitor Analysis Deep Research Prompts for Documentation

A comprehensive collection of research prompts designed for AI-powered deep research tools. Each template includes detailed instructions...

Feb 2 4

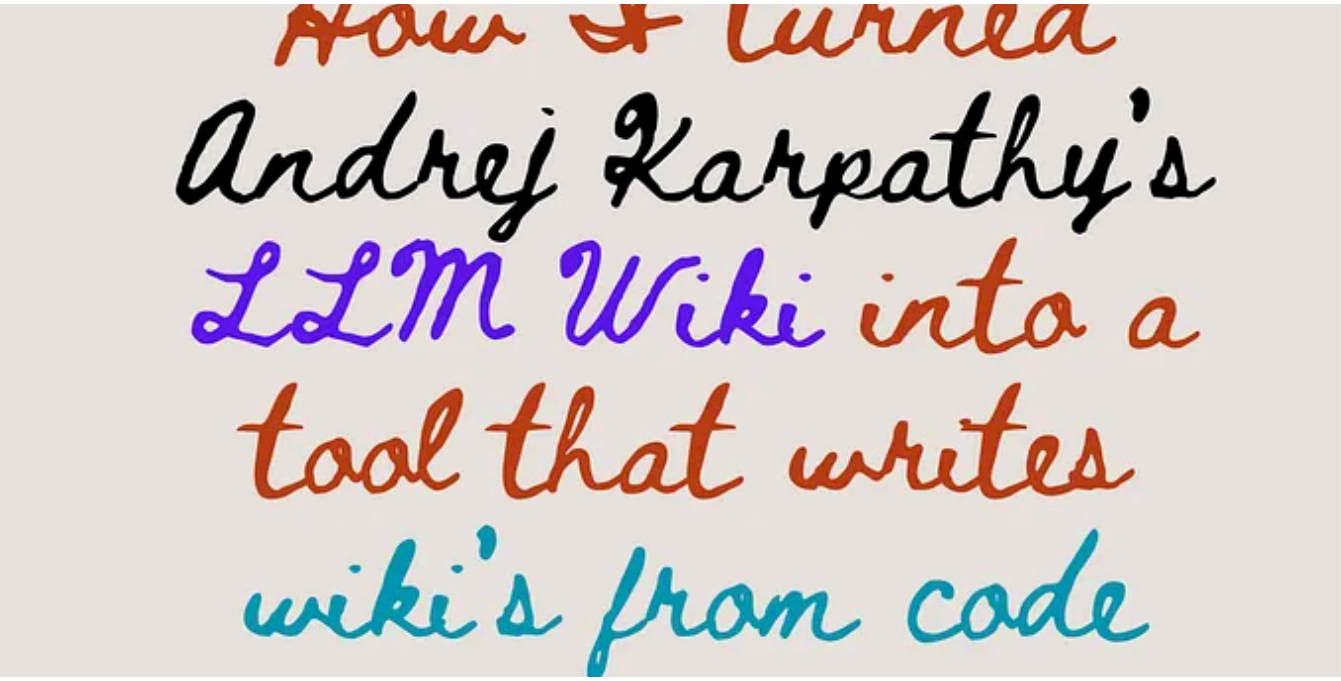


Balu Kosuri

SEO, AEO, and GEO for Technical Writers

Imagine you have built the most beautiful library in the world. Every shelf is perfectly organized. Every book is accurate, up to date, and...

Mar 9 3 2



Balu Kosuri

How I turned Andrej Karpathy's LLM Wiki into a tool that writes wiki's from code

By Balu Kosuri

2d ago 8

See all from Balu Kosuri

Recommended from Medium

In Vibe Coding by Alex Dunlop

I Cut Claude Code's Output Tokens by 75%. Why Did Nobody Tell Me?

Fewer tokens. Somehow more accurate. There's a paper.

Apr 11 1.2K 15

E2B	2.3B	4GB	85	6.1/10	6.5/10	On-device, edge
E4B	4.5B	8GB	38	7.4/10	7.8/10	Local dev, laptops
26B MoE	**3.8B active**	**18GB**	**42**	**9.1/10**	**8.9/10**	**Most use cases**

In Towards AI by Chew Loong Nian - AI ENGINEER

I Tested All 4 Gemma 4 Models: The 26B One Is Cheating (In the Best Way)

Google dropped Gemma 4 this week under Apache 2.0 — free to use, free to commercialize, no strings attached. Four model sizes landed at...

Apr 4 233 3

In Data Science Collective by Gao Dalie ( )

Gemma 4 + Turboquant + RAG: Better OCR & Self-Hosted

If you don't have a Medium subscription, use this link to read the full article: Link

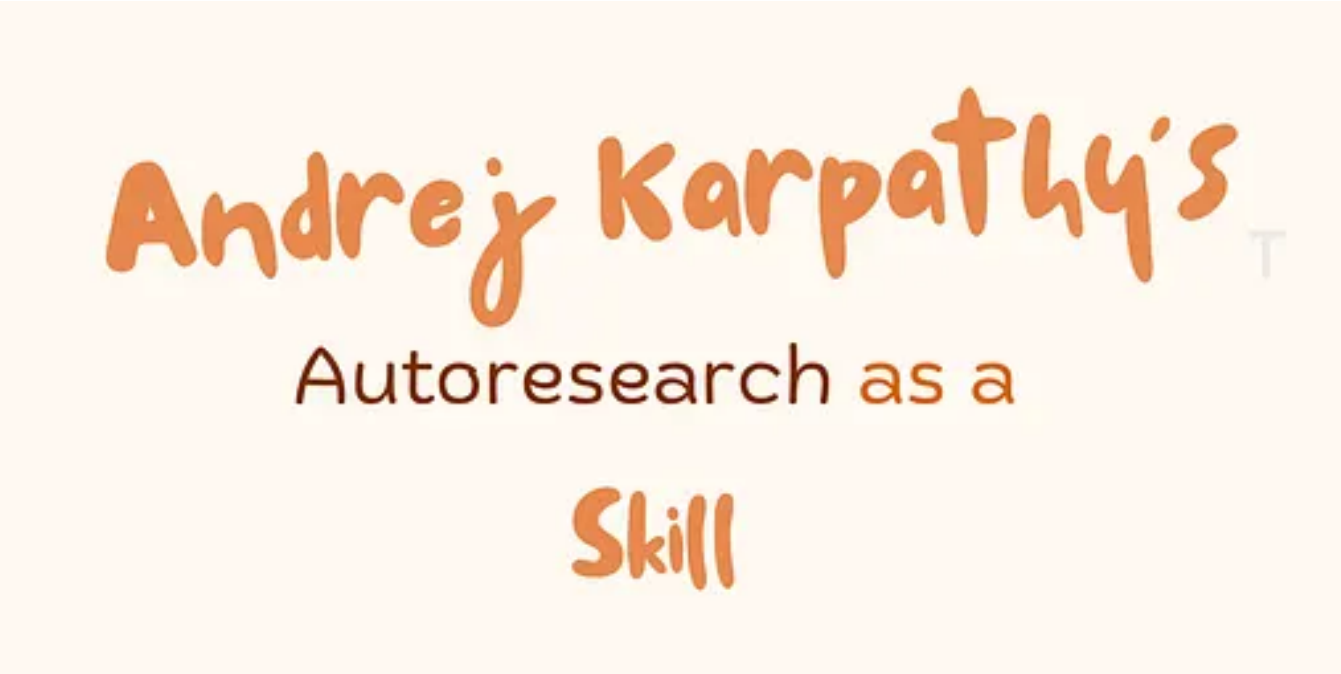
Apr 8 405 1

In The Ai Studio by Ai studio

11 Insane Use Cases of OpenClaw AI

What Happens When You Give an AI Agent Access to Your Entire Digital Life

Mar 28 558 17

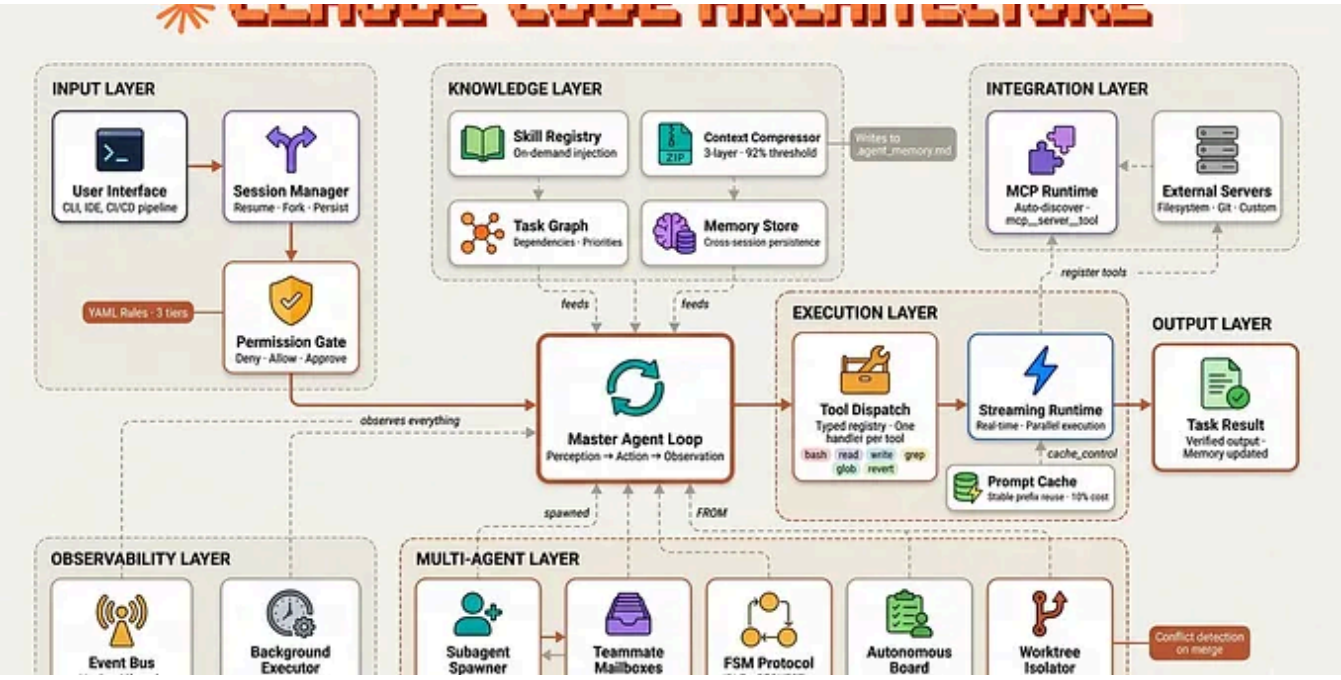


Balu Kosuri

I Turned Andrej Karpathy's Autoresearch Into a Universal Skill

By Balasubramanyam Kosuri

Mar 21 504 9



In Level Up Coding by Fareed Khan

Building Claude Code with Harness Engineering

Multi-agents, MCP, skills system, context pipelines and more

Apr 6 1.3K 9

See more recommendations